

How to Use This Coding Guide

The coding round is reported to contain about 2 LeetCode-easy problems.

The problems below cover the most common easy patterns: hash maps, stacks, two pointers and single-pass scans.

Every Python solution has been executed and verified against its sample output.

Coding Problems

Problem 1: Two Sum

Difficulty: *Easy* | Pattern: *Hash Map*

Problem Statement

Given an array of integers and a target value, return the indices of the two numbers that add up to the target.

Input Format

Line 1: the array. Line 2: the target.

Output Format

The two indices, space-separated.

Constraints

$2 \leq n \leq 10^4$; exactly one valid answer exists.

Sample Input

```
3 2 4
6
```

Sample Output

```
1 2
```

Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i

i, j = two_sum([3, 2, 4], 6)
print(i, j)
```

Step-by-Step Explanation

Store each number's index while scanning. If the complement ($\text{target} - x$) was already seen, return the stored index and the current index. $2 + 4 = 6$.

Dry Run

```
i=0 x=3 need=3 not seen; store 3->0
i=1 x=2 need=4 not seen; store 2->1
i=2 x=4 need=2 seen at 1 -> (1,2)
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 2: Valid Parentheses

Difficulty: *Easy* | Pattern: *Stack*

Problem Statement

Given a string of brackets '()[]{}', determine if it is valid: brackets must close in the correct order.

Input Format

A single line: the bracket string.

Output Format

'True' if valid, otherwise 'False'.

Constraints

1 <= length <= 10⁴

Sample Input

```
()[]{}
```

Sample Output

```
True
```

Python Solution

```
def is_valid(s):
    pairs = {'(': ')', '[': ']', '{': '}'}
    stack = []
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(is_valid('()[]{}'))
```

Step-by-Step Explanation

Push opening brackets onto a stack. For each closing bracket, the top of the stack must be its matching opener. At the end the stack must be empty.

Dry Run

```
( push -> ['(']
) pop '(' matches
[ push
] pop '['
{ push
} pop '{'
stack empty -> True
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 3: Contains Duplicate

Difficulty: *Easy* | Pattern: *Set*

Problem Statement

Given an integer array, return True if any value appears at least twice, and False if every element is distinct.

Input Format

First line: n. Second line: the n integers.

Output Format

'True' or 'False'.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
5
1 2 3 1 4
```

Sample Output

```
True
```

Python Solution

```
def contains_duplicate(nums):
    return len(set(nums)) != len(nums)

print(contains_duplicate([1, 2, 3, 1, 4]))
```

Step-by-Step Explanation

A set removes duplicates. If the set is smaller than the list, at least one duplicate existed. Here the list has 5 items but only 4 distinct values.

Dry Run

```
set([1,2,3,1,4]) = {1,2,3,4} size 4
list size 5
4 != 5 -> True
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 4: Best Time to Buy and Sell Stock

Difficulty: *Easy* | Pattern: *Single Pass / Running Minimum*

Problem Statement

Given daily prices, find the maximum profit from one buy and one later sell. If no profit is possible, return 0.

Input Format

First line: n. Second line: the n prices.

Output Format

The maximum profit.

Constraints

$1 \leq n \leq 10^5$

Sample Input

```
6
7 1 5 3 6 4
```

Sample Output

```
5
```

Python Solution

```
def max_profit(prices):
    min_price = float('inf')
    best = 0
    for p in prices:
        min_price = min(min_price, p)
        best = max(best, p - min_price)
    return best

print(max_profit([7, 1, 5, 3, 6, 4]))
```

Step-by-Step Explanation

Track the lowest price seen so far. At each day the best profit is the current price minus that minimum. Buy at 1, sell at 6, profit 5.

Dry Run

```
7: min=7 best=0
1: min=1 best=0
5: best=4
3: best=4
6: best=5
4: best=5
Result = 5
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 5: Valid Anagram

Difficulty: *Easy* | Pattern: *Frequency Counting*

Problem Statement

Given two strings, determine whether one is an anagram of the other (same characters with the same counts).

Input Format

Line 1: first string. Line 2: second string.

Output Format

'True' or 'False'.

Constraints

$1 \leq \text{length} \leq 10^4$

Sample Input

```
listen
silent
```

Sample Output

```
True
```

Python Solution

```
def is_anagram(a, b):
    if len(a) != len(b):
        return False
    counts = {}
    for ch in a:
        counts[ch] = counts.get(ch, 0) + 1
    for ch in b:
        if counts.get(ch, 0) == 0:
            return False
        counts[ch] -= 1
    return True

print(is_anagram('listen', 'silent'))
```

Step-by-Step Explanation

Count characters of the first string, then decrement for the second. If any character runs out or lengths differ, they are not anagrams. 'listen' and 'silent' share the same letters.

Dry Run

```
counts(listen)={l,i,s,t,e,n each 1}
decrement with silent -> all reach 0
Result = True
```

Time Complexity: **O(N)** | Space Complexity: **O(1) fixed alphabet**

Problem 6: Merge Two Sorted Arrays

Difficulty: *Easy* | Pattern: *Two Pointers*

Problem Statement

Given two sorted arrays, merge them into a single sorted array.

Input Format

Line 1: first sorted array. Line 2: second sorted array.

Output Format

The merged sorted array, space-separated.

Constraints

$0 \leq \text{sizes} \leq 10^5$

Sample Input

```
1 3 5
```

2 4 6

Sample Output

1 2 3 4 5 6

Python Solution

```
def merge_sorted(a, b):
    i = j = 0
    merged = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            merged.append(a[i]); i += 1
        else:
            merged.append(b[j]); j += 1
    merged.extend(a[i:])
    merged.extend(b[j:])
    return merged

print(' '.join(map(str, merge_sorted([1, 3, 5], [2, 4, 6]))))
```

Step-by-Step Explanation

Two pointers walk both arrays, always taking the smaller front element. When one array is exhausted, append the rest of the other. This is the merge step of merge sort.

Dry Run

```
1<=2 take1
3>2 take2
3<=4 take3
5>4 take4
5<=6 take5
a done, extend b: 6
Result = 1 2 3 4 5 6
```

Time Complexity: **$O(m + n)$** | Space Complexity: **$O(m + n)$**

BI3 Technologies

Coding Problems (LeetCode-Easy)

Applied Role(s):

Data Engineer - Trainee

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

Table of Contents

How to Use This Coding Guide	1
Coding Problems	1